

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«КАЗАНСКИЙ (ПРИВОЛЖСКИЙ) ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт Вычислительной математики и информационных технологий

Отчет
учебной практики (научно-исследовательская работа (получение первичных
навыков научно-исследовательской работы))

Обучающийся Филадельфов Тихон Георгиевич гр.09-535 _____
(ФИО студента) (Группа) (Подпись)

Научный руководитель:
Доцент кафедры САИТ Васильев А.В. _____
(Подпись)

Руководитель практики от КФУ:

Доцент кафедры САИТ Андрианова А.А.

Оценка за практику _____
(Подпись)

Дата сдачи отчета «____» _____ 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. КЛАССИЧЕСКИЕ НЕЙРОННЫЕ СЕТИ И ИХ ОБУЧЕНИЕ.....	4
2. КВАНТОВЫЕ И ГИБРИДНЫЕ НЕЙРОННЫЕ СЕТИ В PENNYLANE.....	6
3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И МЕТОДИКА ЭКСПЕРИМЕНТОВ.....	8
4. РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ.....	10
ЗАКЛЮЧЕНИЕ.....	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	17
ПРИЛОЖЕНИЕ.....	19

ВВЕДЕНИЕ

Учебная практика (научно-исследовательская работа (получение первичных навыков научно-исследовательской работы)) проходила на кафедре системного анализа и информационных технологий Института вычислительной математики и информационных технологий КФУ с 09 февраля 2026 по 06 июня 2026 года.

Тема работы связана с разработкой библиотеки для работы с квантовыми гибридными нейронными сетями. В первом семестре был выполнен анализ предметной области и существующих программных инструментов. Во втором семестре работа перешла в практическую плоскость: необходимо было разобраться, как устроены и обучаются классические нейросетевые модели, которые служат точкой отсчёта для любых сравнений, и на практике проверить, как ведут себя квантовая и гибридная модели, построенные средствами библиотеки PennyLane [13].

Цель работы — исследовать классические нейросетевые модели классификации и сравнить их с квантовой и гибридной моделями PennyLane по качеству и времени обучения. Для достижения цели решались следующие задачи: изучить устройство и методы обучения многослойного перцептрона; исследовать влияние архитектуры сети, скорости обучения и функции активации на качество классификации; изучить принципы построения параметризованных квантовых схем и их интеграции с PyTorch; реализовать программу для обучения и сравнения моделей; провести эксперименты на пяти наборах данных с многократными запусками и проанализировать результаты.

Результаты этого этапа определяют направление дальнейшей работы над магистерской диссертацией: они показывают, где проходит граница возможностей существующих инструментов и какие требования должна учитывать собственная библиотека.

1. Классические нейронные сети и их обучение

Многослойный перцептрон (MLP) — последовательность полносвязных слоёв. Каждый слой вычисляет линейное преобразование входного вектора с обучаемыми весами и смещениями, после чего применяется нелинейная функция активации [1, 2]. Без нелинейности несколько слоёв сводились бы к одному линейному преобразованию. В работе рассматриваются две функции активации: ReLU, $\max(0, x)$, стандартная для скрытых слоёв, и гиперболический тангенс $\tanh$, ограничивающий выход интервалом $(-1, 1)$.

Последний слой выдаёт по одному числу на класс. Функция потерь — перекрёстная энтропия между распределением, полученным из этих чисел функцией `softmax`, и истинной меткой. Обучение состоит в минимизации потерь градиентными методами: градиенты по всем параметрам вычисляются методом обратного распространения ошибки, который в PyTorch реализован механизмом автоматического дифференцирования `autograd` [12]. Для обновления параметров используется оптимизатор Adam [10], подбирающий величину шага для каждого параметра по оценкам первого и второго моментов градиента. Данные подаются пакетами по 32 объекта, один проход по обучающей выборке называется эпохой.

Чтобы модель не переобучалась, выборка делится на обучающую, валидационную и тестовую части. Параметры обучаются на первой, по второй выбирается момент остановки: если потери на валидации не уменьшаются заданное число эпох подряд, обучение прекращается и восстанавливаются веса с наилучшей валидацией (ранняя остановка) [2, 3]. Тестовая часть используется только для итоговой оценки. Признаки предварительно стандартизуются: приводятся к нулевому среднему и единичной дисперсии, причём параметры стандартизации вычисляются только по обучающей части [4].

Качество классификации оценивается четырьмя метриками [4, 5]. Accuracy — доля верных ответов. Precision класса — доля верных среди объектов, отнесённых моделью к этому классу, recall — доля найденных объектов класса, F1-score — их гармоническое среднее. Для многоклассовых задач precision, recall и F1 усредняются по классам с равными весами (macro-усреднение), поэтому ошибки на редких классах не маскируются частыми.

Качество перцептрона зависит от гиперпараметров: числа и ширины скрытых слоёв, скорости обучения, функции активации [3, 6]. Слишком маленькая сеть не способна описать нелинейную границу классов, слишком большая на малых данных склонна к переобучению. Слишком малая скорость обучения замедляет сходимость, слишком большая делает её неустойчивой. Эти зависимости исследуются в разделе 4.1.

2. Квантовые и гибридные нейронные сети в PennyLane

Состояние кубита описывается двумя комплексными амплитудами α и β : $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, $|\alpha|^2 + |\beta|^2 = 1$. При измерении кубит принимает значение 0 с вероятностью $|\alpha|^2$ и 1 с вероятностью $|\beta|^2$ [8, 9]. Выходом квантовой схемы в работе служит среднее значение наблюдаемой Z для каждого кубита: $\langle Z \rangle = P(0) - P(1)$, число от -1 до 1 . Состояние n кубитов описывается 2^n амплитудами, поэтому моделирование схем на классическом компьютере требует экспоненциально растущих ресурсов.

Квантовая схема строится из вентилях. Вращения $RX(\theta)$, $RY(\theta)$, $RZ(\theta)$ поворачивают состояние кубита на угол θ , двухкубитный вентиль CNOT инвертирует целевой кубит, если управляющий находится в состоянии 1, и создаёт запутанность между кубитами. Если часть углов вращения задаётся входными данными, а часть является обучаемыми параметрами, схема становится обучаемой функцией — параметризованной, или вариационной, квантовой схемой [14, 17].

В библиотеке PennyLane [13] схема описывается функцией Python — квантовым узлом (QNode), который исполняется на симуляторе `default.qubit`. В работе используются два стандартных шаблона. `AngleEmbedding` кодирует признаки в углы вращений RY , по одному признаку на кубит. `StronglyEntanglingLayers` — вариационный слой: произвольное вращение Rot (три обучаемых угла) на каждом кубите и кольцо вентилях CNOT; слой повторяется L раз. Обёртка `qml.qnn.TorchLayer` превращает узел в обычный модуль `PyTorch`, так что квантовую схему можно вставить в нейронную сеть как слой, а её параметры обучаются тем же оптимизатором. Градиенты по параметрам схемы в симуляторе вычисляются обратным распространением, а на реальном квантовом устройстве — по правилу сдвига параметра, требующему двух дополнительных запусков схемы на каждый параметр [15].

На основе этих средств построены две модели (рисунок 1). Квантовая нейронная сеть (Quantum QNN) получает признаки, сжатые методом главных компонент до числа кубитов, кодирует их в схему на 4 кубитах с двумя вариационными слоями и линейно отображает четыре значения $\langle Z \rangle$ в классы. Гибридная модель (Hybrid QNN) содержит классический энкодер — линейный слой, отображающий все признаки в четыре числа, которые функцией $(\pi/2) \cdot \tanh$ переводятся в углы кодирования, — ту же квантовую схему и выходной линейный слой, как в работе [16]. Энкодер обучается вместе со схемой, поэтому гибридная модель не ограничена требованием, чтобы число признаков совпадало с числом кубитов.

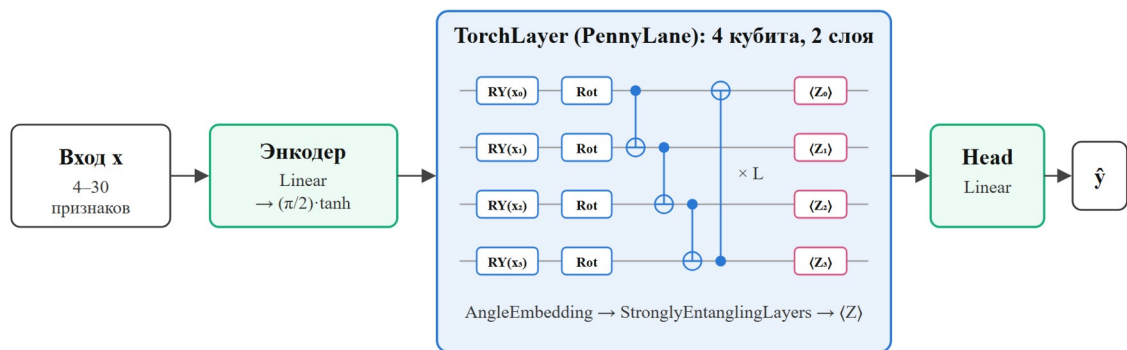


Рисунок 1 – Гибридная модель Hybrid QNN на основе PennyLane

3. Программная реализация и методика экспериментов

Программа написана на Python с использованием PyTorch [12], PennyLane [13] и Scikit-Learn [11]. Этапы работы вынесены в отдельные модули (таблица 1), общая схема показана на рисунке 2. Все модели обучаются одной и той же процедурой, поэтому различия в результатах связаны с архитектурой модели, а не с условиями запуска.

Таблица 1 – Модули программы

Модуль	Назначение
data.py	Загрузка наборов данных, стратифицированное разбиение, стандартизация, PCA
training.py	Обучение с ранней остановкой, расчёт accuracy, precision, recall, F1-score
sem2_experiments.py	Модели MLP, Quantum QNN и Hybrid QNN, описание серий экспериментов, параллельный запуск, сохранение результатов
sem2_analysis.py	Сводные таблицы, критерий Уилкоксона, построение графиков

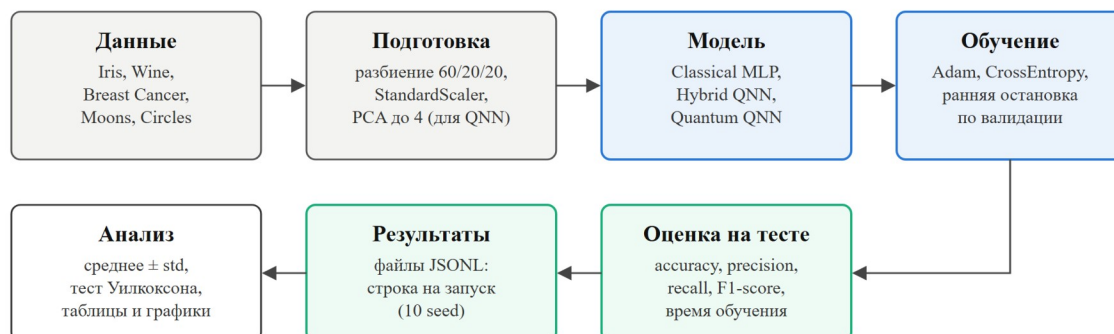


Рисунок 2 – Схема экспериментального пайплайна

Эксперименты проводились на пяти табличных наборах данных (таблица 2). Iris, Wine и Breast Cancer встроены в Scikit-Learn [11]; Moons и Circles — синтетические двумерные задачи, для решения которых нужна нелинейная граница между классами.

Таблица 2 – Наборы данных

Набор данных	Объектов	Признаков	Классов
Iris	150	4	3
Wine	178	13	3

Набор данных	Объектов	Признаков	Классов
Breast Cancer	569	30	2
Moons	400	2	2
Circles	400	2	2

Данные делились стратифицированно на обучающую, валидационную и тестовую выборки в пропорции 60/20/20. Для Quantum QNN признаки Wine и Breast Cancer сжимались методом главных компонент до четырёх; у Iris признаков ровно четыре, а два признака Moons и Circles кодируются на первые два кубита. Обучение: Adam со скоростью 0,01, пакет 32 объекта, до 300 эпох с ранней остановкой после 30 эпох без улучшения валидационных потерь.

Первый вариант эксперимента, выполненный в середине семестра, содержал по одному запуску каждой модели с фиксированным числом эпох. Результаты такого сравнения оказались неустойчивыми: смена случайного разбиения меняла порядок моделей. Поэтому методика была расширена: каждая конфигурация запускается с десятью значениями seed (0–9), который задаёт и разбиение данных, и начальные веса модели, а модели сравниваются на одинаковых seed парным критерием Уилкоксона на уровне значимости 0,05 [7]. Всего проведено четыре серии экспериментов (таблица 3). Запуски выполнялись параллельно в 11 процессах на ноутбуке с процессором Intel Core i7-9750H.

Таблица 3 – Серии экспериментов

Серия	Что варьируется	Запусков
Архитектура MLP	6 конфигураций скрытых слоёв $\times$ 5 наборов	300
Параметры обучения	скорость обучения 0,001 / 0,01 / 0,1 $\times$ ReLU и tanh $\times$ 5 наборов	300
Сравнение с PennyLane	MLP 32–16, MLP равного размера, Hybrid QNN, Quantum QNN $\times$ 5 наборов	200
Время обучения	Hybrid QNN на 2–10 кубитах и MLP, 20 эпох	18

4. Результаты экспериментов

4.1 Архитектура перцептрона

В первой серии исследовалось влияние архитектуры перцептрона: от одного скрытого слоя из 8 нейронов (266 параметров на Breast Cancer) до трёх слоёв 64–32–16 (4626 параметров). Результаты приведены в таблице 4 и на рисунке 3.

Таблица 4 – Ассигасу перцептрона при разной архитектуре (среднее по 10 запускам)

Скрытые слои	Парам. (BC)	Iris	Wine	Breast Cancer	Moons	Circles	Среднее
8	266	0,960	0,969	0,977	0,931	0,993	0,966
32	1058	0,963	0,983	0,979	0,936	0,995	0,971
128	4226	0,953	0,986	0,975	0,931	0,996	0,969
32–16	1554	0,957	0,981	0,976	0,930	0,995	0,968
64–32	4130	0,943	0,986	0,977	0,936	0,994	0,967
64–32–16	4626	0,953	0,981	0,976	0,938	0,995	0,969

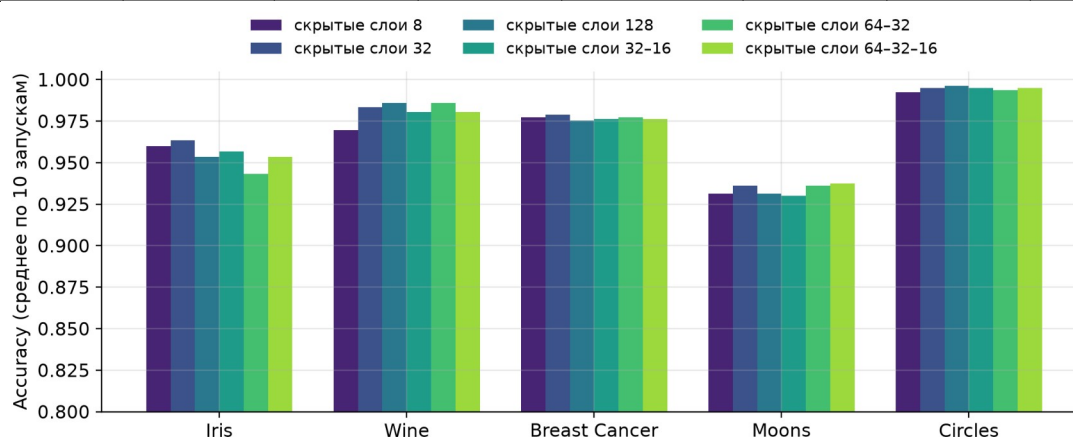


Рисунок 3 – Ассигасу перцептрона при разной архитектуре скрытых слоёв

Различия между архитектурами малы: средняя по наборам ассигасу лежит в пределах 0,966–0,971, что сопоставимо с разбросом между запусками. Заметно уступает только самая маленькая сеть на наборе Wine (0,969 против 0,981–0,986 у остальных): восьми нейронов недостаточно для 13 признаков и трёх классов. Увеличение сети сверх 32 нейронов не повышает качество, но

ускоряет сходимость: ранняя остановка срабатывает в среднем через 160 эпох у сети из 8 нейронов и через 60 эпох у сети 64–32–16. Для небольших табличных наборов достаточно перцептрона с одним-двумя скрытыми слоями, а дальнейший рост числа параметров не окупается.

4.2 Скорость обучения и функция активации

Во второй серии для перцептрона 32–16 менялись скорость обучения (0,001, 0,01 и 0,1) и функция активации (таблица 5). Малый шаг 0,001 даёт наибольшую среднюю точность (0,971 для ReLU), но требует примерно в 2,4 раза больше эпох, чем шаг 0,01 (194 против 80). Большой шаг 0,1 ускоряет остановку (56 эпох), но снижает качество, заметнее всего на Wine (0,967 против 0,986): обучение становится менее устойчивым. Функция активации на результат почти не влияет — различия ReLU и tanh не превышают одного процентного пункта. В остальных сериях использовались ReLU и шаг 0,01 как компромисс между качеством и временем обучения.

Таблица 5 – Ассурасу перцептрона 32–16 при разной скорости обучения и функции активации

Активация, шаг	Iris	Wine	Breast Cancer	Moons	Circles	Среднее	Эпох
ReLU, 0,001	0,960	0,986	0,977	0,938	0,996	0,971	194
ReLU, 0,01	0,957	0,981	0,976	0,930	0,995	0,968	80
ReLU, 0,1	0,960	0,967	0,975	0,930	0,990	0,964	56
tanh, 0,001	0,957	0,978	0,980	0,921	0,996	0,966	207
tanh, 0,01	0,957	0,978	0,981	0,929	0,995	0,968	93
tanh, 0,1	0,953	0,972	0,974	0,926	0,991	0,963	57

4.3 Сравнение с квантовой и гибридной моделями PennyLane

В третьей серии перцептроны сравнивались с моделями PennyLane (таблица 6, рисунок 4). Кроме перцептрона 32–16 в сравнение включён перцептрон равного размера — с одним скрытым слоем, ширина которого подобрана так, чтобы число параметров совпадало с гибридной моделью (например, 59 параметров на Iris). Это позволяет отделить влияние архитектуры от влияния размера модели.

Таблица 6 – Сравнение моделей (среднее по 10 запускам)

Набор	Модель	Ассурасу	F1-score	Эпоха, мс	Параметров
Iris	MLP 32–16	$0,957 \pm 0,035$	0,957	8,5	739
	MLP равн. размера	$0,967 \pm 0,035$	0,967	6,3	59
	Hybrid QNN	$0,970 \pm 0,040$	0,970	184,5	59
	Quantum QNN	$0,927 \pm 0,047$	0,926	165,3	39
Wine	MLP 32–16	$0,981 \pm 0,019$	0,981	10,9	1027
	MLP равн. размера	$0,975 \pm 0,016$	0,975	8,2	88
	Hybrid QNN	$0,967 \pm 0,022$	0,967	227,1	95
	Quantum QNN	$0,897 \pm 0,063$	0,899	215,9	39
Breast Cancer	MLP 32–16	$0,976 \pm 0,012$	0,975	28,4	1554
	MLP равн. размера	$0,976 \pm 0,008$	0,975	17,1	167
	Hybrid QNN	$0,982 \pm 0,009$	0,980	610,8	158
	Quantum QNN	$0,895 \pm 0,028$	0,886	548,8	34
Moons	MLP 32–16	$0,930 \pm 0,030$	0,930	21,9	658
	MLP равн. размера	$0,934 \pm 0,021$	0,934	16,9	47
	Hybrid QNN	$0,905 \pm 0,035$	0,905	449,7	46
	Quantum QNN	$0,896 \pm 0,026$	0,896	401,3	34
Circles	MLP 32–16	$0,995 \pm 0,006$	0,995	23,4	658
	MLP равн. размера	$0,994 \pm 0,007$	0,994	14,5	47
	Hybrid QNN	$0,993 \pm 0,013$	0,993	458,4	46
	Quantum QNN	$0,995 \pm 0,006$	0,995	366,2	34

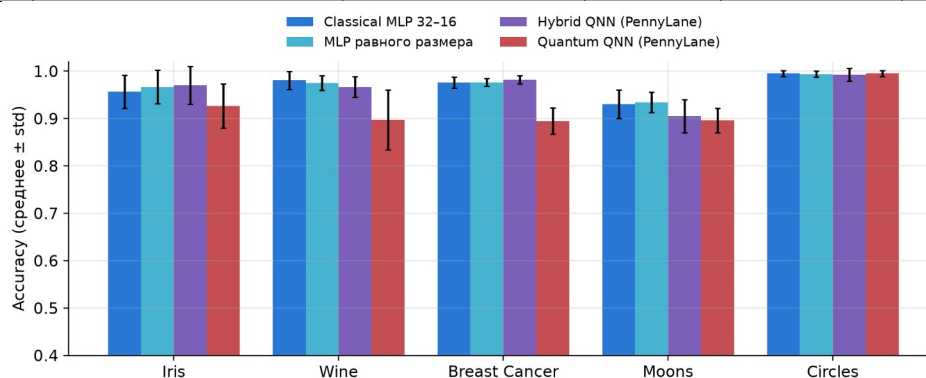


Рисунок 4 – Ассурасу классических моделей и моделей PennyLane (среднее $\pm$ стандартное отклонение)

Гибридная модель показала качество на уровне перцептронов: на Iris и Breast Cancer немного выше (0,970 и 0,982 против 0,957 и 0,976 у перцептрона 32–16), на Wine и Moons немного ниже (0,967 и 0,905 против 0,981 и 0,930).

Ни одно из этих различий не является статистически значимым: по критерию Уилкоксона p от 0,062 до 0,75 как для перцептрона 32–16, так и для перцептрона равного размера.

Чисто квантовая модель значимо уступает гибридной на Iris (на 4,3 п. п., $p = 0,039$), Wine (на 6,9 п. п., $p = 0,016$) и Breast Cancer (на 8,7 п. п., $p = 0,002$). У неё нет обучаемого энкодера: признаки подаются в углы вращений напрямую, а для Wine и Breast Cancer ещё и сжимаются методом главных компонент до четырёх, при этом часть информации теряется. Энкодер гибридной модели сам обучается выбирать нужную комбинацию признаков. На двумерных Moons и Circles сжатие не требуется, и квантовая модель работает на уровне гибридной, а на Circles — на уровне перцептрона (0,995).

4.4 Время обучения

Цена квантовой части — время обучения. Эпоха Hybrid QNN длится от 184 до 611 мс против 8–28 мс у перцептрона 32–16, то есть примерно в 21 раз дольше. Полное обучение одной модели занимает 33–112 с против 1–3 с, в 25–50 раз дольше, так как гибридная модель к тому же обучается больше эпох. Причина — моделирование квантовой схемы на классическом компьютере.

В серии с разным числом кубитов (рисунок 5) время эпохи на Breast Cancer растёт с 0,31 с на двух кубитах до 1,89 с на десяти, что в 64 раза больше, чем у перцептрона (0,029 с). В этом диапазоне рост близок к линейному: вектор состояния ещё мал (не более 1024 амплитуд), и основное время уходит на накладные расходы. При дальнейшем увеличении числа кубитов начнёт сказываться экспоненциальный рост размера состояния. На реальном квантовом устройстве градиенты вычисляются по правилу сдвига параметра, которое требует двух дополнительных запусков схемы на каждый параметр [15], поэтому время обучения будет ещё больше.

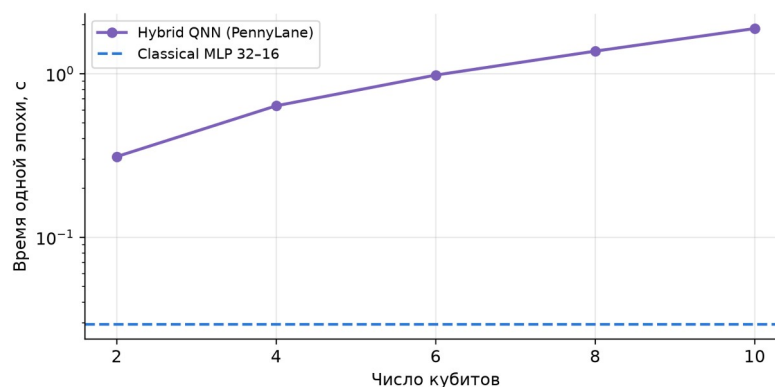


Рисунок 5 – Время одной эпохи Hybrid QNN в зависимости от числа кубитов (Breast Cancer)

Таким образом, на небольших табличных наборах перцептрон с одним-двумя скрытыми слоями достигает высокой точности, а гибридная модель PennyLane не даёт статистически значимого выигрыша ни перед более крупным перцептроном, ни перед перцептроном равного размера, но обучается в десятки раз дольше. Для систематического исследования, требующего тысяч обучений с разными параметрами схемы, такой скорости недостаточно — это определяет задачу следующего этапа: разработку собственной библиотеки с быстрым симулятором квантовых схем.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики была продолжена работа над темой, связанной с разработкой библиотеки для работы с квантовыми гибридными нейронными сетями. Если в первом семестре основное внимание уделялось анализу предметной области, то во втором семестре была выполнена практическая часть: изучены классические нейросетевые модели, реализована программа для обучения и сравнения моделей и проведены эксперименты.

Программа на Python с использованием PyTorch, PennyLane и Scikit-Learn реализует подготовку данных, три типа моделей — многослойный перцептрон, квантовую и гибридную нейронные сети, — единую процедуру обучения с ранней остановкой, расчёт метрик и статистическое сравнение. Проведено 818 обучений на пяти наборах данных, каждая конфигурация запускалась десять раз.

Исследование перцептрона показало, что на небольших табличных наборах качество слабо зависит от архитектуры (средняя accuracy 0,966–0,971), а заметнее влияет скорость обучения. Гибридная модель PennyLane показала качество на уровне перцептронов без статистически значимых различий, чисто квантовая модель уступила гибридной на 4,3–8,7 п. п. на наборах Iris, Wine и Breast Cancer. Обучение гибридной модели в десятки раз медленнее, чем перцептрона.

Полученные результаты определили задачу следующего этапа магистерской работы: разработку собственной библиотеки с быстрым симулятором квантовых схем, которая позволит провести систематическое сравнение гибридных и классических моделей на большем числе конфигураций и наборов данных.

За время практики были реализованы следующие компетенции (таблица 7):

Таблица 7 – Компетенции

Шифр компетенции	Расшифровка приобретаемой компетенции	Расшифровка освоения компетенции
УК-1	Способен осуществлять критический анализ проблемных ситуаций на основе системного подхода, вырабатывать стратегию действий	Проведён анализ классических нейросетевых моделей и инструментов квантового машинного обучения. Выявлены недостатки первого варианта эксперимента (один запуск, фиксированное число эпох) и выработана методика многократных запусков со статистической проверкой.
УК-4	Способен применять современные коммуникативные технологии, в том числе на иностранном(ых) языке(ах), для академического и профессионального взаимодействия	Изучены англоязычные статьи и документация PyTorch, PennyLane и Scikit-Learn; результаты оформлены в виде отчёта с таблицами и графиками.
УК-6	Способен определять и реализовывать приоритеты собственной деятельности и способы ее совершенствования на основе самооценки	Спланированы этапы работы: изучение литературы, реализация программы, серии экспериментов, анализ результатов. Определены задачи следующего этапа магистерской работы.
ОПК-1	Способен находить, формулировать и решать актуальные проблемы прикладной математики, фундаментальной информатики и информационных технологий	Сформулирована задача сравнения классических, квантовых и гибридных моделей классификации; определены критерии сравнения — качество классификации и время обучения.
ОПК-2	Способен применять компьютерные/суперкомпьютерные методы, современное программное обеспечение (в том числе отечественного производства) для решения задач профессиональной деятельности	Разработана программа на Python с использованием PyTorch, PennyLane и Scikit-Learn; эксперименты выполнены параллельно в нескольких процессах.
ОПК-3	Способен проводить анализ математических моделей, создавать инновационные методы решения прикладных задач профессиональной деятельности в области информатики и математического моделирования	Проведён анализ моделей на пяти наборах данных: исследовано влияние архитектуры и параметров обучения перцептрона, выполнено сравнение с квантовой и гибридной моделями, оценена зависимость времени обучения от числа кубитов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) Хайкин, С. Нейронные сети: полный курс / С. Хайкин ; пер. с англ. — 2-е изд. — Москва : Вильямс, 2006. — 1104 с.
- 2) Гудфеллоу, Я. Глубокое обучение / Я. Гудфеллоу, И. Бенджио, А. Курвилль ; пер. с англ. — Москва : ДМК Пресс, 2018. — 652 с.
- 3) Николенко, С. Глубокое обучение. Погружение в мир нейронных сетей / С. Николенко, А. Кадурын, Е. Архангельская. — Санкт-Петербург : Питер, 2018. — 480 с.
- 4) Жерон, О. Прикладное машинное обучение с помощью Scikit-Learn и TensorFlow / О. Жерон ; пер. с англ. — Санкт-Петербург : Альфа-книга, 2018. — 688 с.
- 5) Рашка, С. Python и машинное обучение / С. Рашка ; пер. с англ. — Москва : ДМК Пресс, 2017. — 418 с.
- 6) Шолле, Ф. Глубокое обучение на Python / Ф. Шолле ; пер. с англ. — Санкт-Петербург : Питер, 2018. — 400 с.
- 7) Кобзарь, А. И. Прикладная математическая статистика. Для инженеров и научных работников / А. И. Кобзарь. — Москва : Физматлит, 2006. — 816 с.
- 8) Нильсен, М. А. Квантовые вычисления и квантовая информация / М. А. Нильсен, И. Л. Чанг ; пер. с англ. — Москва : Мир, 2006. — 824 с.
- 9) Прескилл, Дж. Квантовая информация и квантовые вычисления. Т. 1 / Дж. Прескилл ; пер. с англ. — Москва ; Ижевск : НИЦ «Регулярная и хаотическая динамика», 2008. — 464 с.
- 10) Kingma, D. P. Adam: A method for stochastic optimization / D. P. Kingma, J. Ba // 3rd International Conference on Learning Representations (ICLR). — 2015. — URL: <https://arxiv.org/abs/1412.6980> (дата обращения: 28.09.2026).
- 11) Pedregosa, F. Scikit-learn: Machine learning in Python / F. Pedregosa [et al.] // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825–2830.

- 12) Paszke, A. PyTorch: An imperative style, high-performance deep learning library / A. Paszke [et al.] // Advances in Neural Information Processing Systems. — 2019. — Vol. 32.
- 13) Bergholm, V. PennyLane: Automatic differentiation of hybrid quantum-classical computations [Электронный ресурс] / V. Bergholm [et al.]. — URL: <https://arxiv.org/abs/1811.04968> (дата обращения: 28.09.2026).
- 14) Schuld, M. Circuit-centric quantum classifiers / M. Schuld, A. Bocharov, K. M. Svore, N. Wiebe // Physical Review A. — 2020. — Vol. 101. — Art. 032308.
- 15) Schuld, M. Evaluating analytic gradients on quantum hardware / M. Schuld, V. Bergholm, C. Gogolin [et al.] // Physical Review A. — 2019. — Vol. 99. — Art. 032331.
- 16) Mari, A. Transfer learning in hybrid classical-quantum neural networks / A. Mari, T. R. Bromley, J. Izaac [et al.] // Quantum. — 2020. — Vol. 4. — Art. 340.
- 17) Preskill, J. Quantum computing in the NISQ era and beyond / J. Preskill // Quantum. — 2018. — Vol. 2. — Art. 79.

ПРИЛОЖЕНИЕ

Модели, использованные в экспериментах (файл sem2_experiments.py):

```
def mlp(in_dim, n_classes, hidden, act="relu"):
    from torch import nn
    A = {"relu": nn.ReLU, "tanh": nn.Tanh}[act]
    layers, d = [], in_dim
    for h in hidden:
        layers += [nn.Linear(d, h), A()]
        d = h
    layers.append(nn.Linear(d, n_classes))
    return nn.Sequential(*layers)

def pl_layer(n_qubits=4, n_layers=2, diff_method="backprop"):
    import pennylane as qml
    dev = qml.device("default.qubit", wires=n_qubits)

    @qml.qnode(dev, interface="torch", diff_method=diff_method)
    def circuit(inputs, weights):
        qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation="Y")
        qml.StronglyEntanglingLayers(weights, wires=range(n_qubits))
        return [qml.expval(qml.PauliZ(q)) for q in range(n_qubits)]

    return qml.qnn.TorchLayer(circuit, {"weights": (n_layers, n_qubits, 3)})

def pl_hybrid(in_dim, n_classes, n_qubits=4, n_layers=2):
    import torch
    from torch import nn

    class Angles(nn.Module): #  $(\pi/2) \cdot \tanh$ : углы кодирования в  $(-\pi/2, \pi/2)$ 
        def forward(self, x):
            return 0.5 * math.pi * torch.tanh(x)

    return nn.Sequential(nn.Linear(in_dim, n_qubits), Angles(),
        pl_layer(n_qubits, n_layers),
        nn.Linear(n_qubits, n_classes))

def pl_quantum(n_classes, n_qubits=4, n_layers=2):
    from torch import nn
    return nn.Sequential(pl_layer(n_qubits, n_layers), nn.Linear(n_qubits,
        n_classes))
```